Data Structures and Algorithms

1. Lists

A **list** is a linear data structure containing elements arranged in a particular order.

Types of Lists

- **Array/List using contiguous memory**
  - Elements are stored next to each other.
  - Fast access using an index.
  - Example: A[0], A[1], A[2]
- **Linked list**
  - Elements are stored in separate nodes.
  - Each node contains data and a link/reference to another node.

Common List Operations

Operation	Typical Array List	Linked List
Access by index	O(1)	O(n)
Search	O(n)	O(n)
Insert at beginning	O(n)	O(1)
Insert at end	O(1)*	O(n) or O(1)**
Delete at beginning	O(n)	O(1)
Delete at end	O(1)	O(n) or O(1)**

* Amortized for a dynamic array.
** Depends on whether a tail pointer is maintained.

Key idea
Array: fast access, slower insertion/deletion in the middle.
Linked list: slower access, but efficient insertion/deletion when you already have the relevant node/reference.

2. Linked Lists

A **linked list** consists of nodes connected through links/references.
A basic node contains:
[data | next]
Example:
HEAD
↓
[10 | •] → [20 | •] → [30 | NULL]

Singly Linked List
Each node points only to the **next** node.
A → B → C → NULL
Advantages

- Dynamic size
- Easy insertion/deletion
- Does not require contiguous memory

Disadvantages

- No direct/random access
- Extra memory required for pointers/references
- Searching is generally O(n)

Doubly Linked List
Each node has two links:
[prev | data | next]
Example:
NULL ← A ⇄ B ⇄ C → NULL
Advantages:
- Can move forward and backward.
- Easier deletion when a node is known.

Disadvantage:
- Requires more memory because of the additional pointer.

Circular Linked List
The last node points back to the first node.
A → B → C
↑ ↓
└──────┘
There is no NULL at the end.

Important Linked-List Operations

Traversal
Visit every node.
current = HEAD

```
while current != NULL:
   process(current.data)
   current = current.next
```

Time complexity:
O(n)

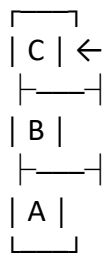
Insert at Beginning
Given:
HEAD → A → B
Insert X:
X → A → B
Steps:
X.next = HEAD
HEAD = X
Complexity: **O(1)**

Delete from Beginning
HEAD = HEAD.next
Complexity: **O(1)**

3. Stack

A **stack** follows:
LIFO — Last In, First Out

The last element inserted is the first one removed.
Think of a stack of plates:



If C was inserted last, C comes out first.

Main Stack Operations

Push
Adds an element to the top.
push(10)
push(20)
push(30)
Stack:
30 ← TOP
20
10

Pop
Removes the top element.
pop()
Returns:
30

Peek / Top
Looks at the top element **without removing it**.
peek() → 20

IsEmpty
Checks whether the stack contains no elements.

Stack Complexity

Operation	Complexity
Push	O(1)
Pop	O(1)
Peek	O(1)
IsEmpty	O(1)
Search	O(n)

Stack Applications
Stacks are commonly used for:

- Function calls
- Recursion
- Undo/redo
- Browser back button

- Expression evaluation
- Parentheses matching
- Depth-First Search (DFS)

Stack Overflow
Occurs when attempting to push onto a **full fixed-size stack**.

Stack Underflow
Occurs when attempting to pop from an **empty stack**.

4. Queue

A **queue** follows:
FIFO — First In, First Out
The first element inserted is the first one removed.
Think of people waiting in line:
FRONT REAR
↓ ↓
[10] → [20] → [30] → [40]
10 leaves first.

Main Queue Operations

Enqueue
Adds an element at the **rear**.
enqueue(50)

Dequeue
Removes an element from the **front**.
dequeue()
Returns the oldest element.

Front / Peek
Returns the front element without removing it.

IsEmpty
Checks whether the queue is empty.

Queue Complexity
With a proper implementation:

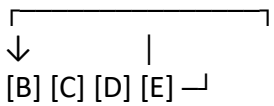
Operation	Complexity
Enqueue	O(1)
Dequeue	O(1)
Peek	O(1)
IsEmpty	O(1)
Search	O(n)

Types of Queues

Simple Queue

FRONT → [A][B][C][D] ← REAR
Insert at rear and remove from front.

Circular Queue
The rear wraps around to the beginning.



Useful for efficiently utilizing fixed-size array space.

Common formula
For an array of size N:
next position = (index + 1) % N

Priority Queue
Elements are removed according to **priority**, not simply arrival order.
Example:
Patient A → priority 2
Patient B → priority 5
Patient C → priority 1
Patient B may be served first because it has the highest priority.

Deque
Deque = Double-Ended Queue
Insertion and deletion can occur at both ends.
← [A] [B] [C] [D] →

5. Hashing

Hashing is a technique used to store and retrieve data efficiently using a **hash table**.
The basic idea:
Key → Hash Function → Index
Example:
key = 25

hash(key) = 25 % 10
= 5
Therefore, key 25 is stored at index 5.

Hash Table
Suppose we have a table of size 10:

Index	Value
0	-
1	-
2	-
3	-
4	-
5	25
6	-
7	-
8	-
9	-

The hash function determines where a key should be stored.
A common hash function is:
h(k) = k % table_size

Collision

A **collision** occurs when two different keys produce the same hash index.
Example:
h(25) = 25 % 10 = 5
h(35) = 35 % 10 = 5
Both want index 5.
Therefore:
25 —→ 5
35 —→ 5
This is a collision.

Collision Resolution

A. Separate Chaining
Each table position stores a linked list of elements.
Index 5:

5 → [25] → [35] → [45]
Advantages:

- Simple
- Can store multiple elements at one index

B. Open Addressing
Instead of using linked lists, find another empty location in the table.
Linear Probing
If index 5 is occupied:
5 → 6 → 7 → 8 ...
Formula:
h(k, i) = (h(k) + i) % m
where:

- k = key
- i = attempt number
- m = table size

Quadratic Probing
Instead of checking consecutive positions:
5 → 6 → 9 → ...
A common formula is:
$h(k, i) = (h(k) + i^2) \% m$

Hashing Complexity
With a good hash function and reasonable load factor:

Operation	Average	Worst Case
Search	O(1)	O(n)
Insert	O(1)	O(n)
Delete	O(1)	O(n)

The worst case can occur when many keys collide.

Load Factor
The **load factor** measures how full a hash table is.
α = number of elements / table size
Example:
Elements = 7
Table size = 10

α = 7/10
α = 0.7
A high load factor generally means more collisions and slower operations.

Quick Comparisons

Data Structure	**Main Principle**	**Access**	**Insert/Delete**
Array/List	Indexed sequence	Fast: O(1)	Can be O(n)
Linked List	Nodes + links	O(n)	O(1) at known position
Stack	LIFO	Top only	O(1)
Queue	FIFO	Front/rear	O(1)
Hash Table	Key → hash → index	Avg. O(1)	Avg. O(1)

Easy Memory Tricks

Remember these:
Stack → LIFO → Last enters, first leaves
Queue → FIFO → First enters, first leaves
Linked List → Nodes + Links
Hashing → Key → Hash Function → Index
Collision → Same index
Chaining → Linked list at an index
Linear probing → Check next slot
Circular queue → Wrap around